

Debugging & Problem-Solving Field Guide

A reusable way of thinking about problems, built from real bugs you found and fixed in your own EEG pipeline — not abstract advice, but principles earned from your own work.

Every principle below is illustrated with something that actually happened while building this project. The code details won't always matter years from now — the way of thinking will. That's the actual point of this document.

Principle 1: Verify the fix is actually running before debugging further

When a change you made appears to have no effect at all, the very first thing to check isn't "why didn't this work" — it's "did this code actually run?" A shocking number of debugging sessions end the moment someone realizes they were staring at output from an old version of the file.

Case from your own work — the phantom fix

After adding subject-wise normalization to fix a classifier that was guessing 'seizure' for everyone, the results came back *identical*, down to the decimal, to before the fix. Rather than assuming the fix was wrong, the exact identical numbers were treated as a clue — and it turned out several new lines had simply never been pasted into the working file.

The transferable lesson: When a fix changes nothing at all, suspect that it isn't running before you suspect that it's wrong. Identical output is itself a data point — it usually means nothing changed upstream, not that your reasoning was flawed.

Principle 2: An error message tells you two things: what, and where — read both

Python error and warning output isn't just a message — the accompanying file path and line number are pointing you directly at the exact call that triggered it. Treat that location as part of the information, not decoration to skip past.

Case from your own work — the duplicate channel warning

A warning read: '*Channel names are not unique, found duplicates for: T8-P8*', followed by the exact line that loaded the file. That second line wasn't a separate message — it was Python naming the specific function call responsible, so the real cause (some files genuinely contain two identically-named electrodes) could be tracked down precisely.

The transferable lesson: Before searching online for an error message, read the location it points to in your own code first. Often the fix is obvious once you see exactly which line and which call produced it.

Principle 3: A result that looks too clean or too uniform deserves suspicion, not relief

Real-world results are usually messy. If every single measurement comes back nearly identical, or a metric lands on a suspiciously round number, that pattern is itself informative — it often signals that something structural is happening, not that you got lucky.

Case from your own work — the 24 identical folds

A classifier's evaluation showed *almost the same numbers repeated across 20+ separate patients* (acc=0.50, prec=0.50, rec=1.00). Rather than accepting this as a working model, the uniformity itself was the clue: this is the exact mathematical signature of a model predicting the same class for every single input, regardless of the data.

The transferable lesson: When results look unusually uniform or suspiciously good, work out what pattern a broken or degenerate process would produce, and check whether your numbers match that pattern instead.

Principle 4: When a summary number confuses you, print the raw values underneath it

Aggregate metrics (accuracy, mean, an average score) compress away the exact information you need to diagnose a problem. The fix is almost always to look one level closer to the raw data.

Case from your own work — checking what the model actually predicted

Instead of continuing to theorize about why accuracy looked strange, a single diagnostic line was added: printing the actual count of each predicted label per fold. This immediately confirmed, with certainty rather than guesswork, that the model was outputting only one class for every test point.

The transferable lesson: A temporary print statement showing raw values is one of the highest-value debugging tools there is. Reach for it before reaching for a new theory.

Principle 5: Compare things using the right ruler — absolute scale can lie to you

Not all numbers that look small are actually 'basically zero,' and not all numbers that look huge are actually meaningful. Different quantities can have wildly different natural scales, and comparing them on an absolute scale can produce a completely wrong conclusion.

Case from your own work — the false 'flat features' alarm

A diagnostic flagged 90 out of 108 features as having 'near-zero variance,' which looked alarming. But those 90 features (band power) are naturally tiny numbers ($\sim 1e-10$) for physical reasons unrelated to whether they're informative, while the other 18 (line length) are naturally much larger.

Switching to a scale-independent measure (coefficient of variation: std divided by mean) showed the original alarm was a false one.

The transferable lesson: When judging whether a number is 'big' or 'small,' 'flat' or 'varying,' ask what that quantity's natural scale actually is before drawing a conclusion. When comparing across different kinds of measurements, use a relative or normalized measure, not a fixed absolute threshold.

Principle 6: Real-world data will violate assumptions your code quietly made

Code written against a dataset's documentation will eventually meet a file that doesn't follow it. This isn't a sign you did something wrong — it's the normal condition of working with real, human-collected data, and it needs to be designed for, not just patched after the fact.

Case from your own work — four separate real dataset quirks

In one dataset (CHB-MIT), four genuine irregularities surfaced through direct use, not documentation: two channels sharing one name in some files, different patients having different numbers of electrodes entirely, one patient recorded with a structurally different setup for part of their sessions, and a non-standard filename marking a split recording.

The transferable lesson: Write code that expects real data to disagree with the spec sheet — validate assumptions explicitly (like checking for required channels before using them) and decide deliberately how to handle violations, rather than discovering them only when something crashes.

Principle 7: Bugs that don't crash are often about control flow, not logic

Some of the most confusing bugs produce no error message at all — the program runs to completion and looks fine, but does less work than it should. These are frequently caused by indentation or scope mistakes: a line sitting one level inside or outside where it belongs.

Case from your own work — the loop that stopped after one iteration

A cross-validation function was supposed to test 24 different patients, one at a time, but only ever printed a result for the first one. Nothing crashed. The cause was two lines meant to run *after* the loop finished — sitting, by indentation, *inside* it instead, causing the function to exit early.

The transferable lesson: When a loop or process runs fewer times than you expect, with no error at all, check indentation and scope before checking your logic. Count what you got versus what you expected, and treat any mismatch as a real signal, not a rounding error.

Principle 8: Change one thing, verify it, before changing the next

Stacking several unverified changes together makes it impossible to know which one actually fixed (or broke) anything. Testing after each individual change keeps every step honest.

Case from your own work — testing every fix against a small synthetic case first

Each fix in this project — the channel-matching logic, the duplicate-channel handling, the class balancing — was verified against a small, fast, synthetic test built specifically to exercise that one change, before ever running it against the full real dataset.

The transferable lesson: Before running a slow, complex process end-to-end, build the smallest possible test that could prove your specific fix right or wrong. It's faster in total, even though it feels like an extra step in the moment.

Principle 9: Update your belief the moment the evidence disagrees with it — say so plainly

A hypothesis is a starting point for investigation, not a commitment to defend. The moment real data contradicts your theory, the correct move is to say so directly and follow the data, not to look for a reason the theory might still be right.

Case from your own work — abandoning the domain-shift theory

A theory that per-patient amplitude differences explained a broken classifier predicted that standardizing features should fix it. When the results came back completely unchanged, that theory was dropped immediately and openly, rather than defended — leading directly to the correct next diagnostic instead of wasted time.

The transferable lesson: Treat a surprising result as more informative than a comforting one. Being visibly wrong quickly is far more useful than being quietly, slowly wrong.

Bonus: the same mindset applies to Git

Every Git problem encountered today followed this same pattern: read the actual message calmly, identify exactly what it's telling you, and act on that — not on panic.

- **'Rejected — fetch first':** not a failure, just information that the remote has a commit your computer doesn't. The fix is always to pull before pushing again.
- **A merge conflict:** Git is explicitly asking you to choose between two versions it found. The <<<<<< and >>>>>> markers are not corruption — they are the exact two versions, shown side by side, waiting for a decision.
- **A confusing terminal screen (Vim) opening unexpectedly:** reading what's actually on screen, rather than assuming something broke, is usually enough to figure out the way out.

A short checklist for next time something looks wrong

- Did I actually run the version of the code I think I did?
- What does the error message say, and — separately — where does it say it happened?
- Does this result look suspiciously uniform, perfect, or round? What would a broken process look like, and does this match that?
- Have I looked at the raw values, not just a summary number?
- Am I comparing two things on a scale that's actually fair to both?
- What assumption about the data might real-world messiness be violating right now?
- If nothing crashed but the output is wrong, have I checked indentation and loop scope?
- Have I verified this specific change on its own, before combining it with anything else?
- Am I defending my first theory, or genuinely updating it based on what just happened?